

Report for the Course on Cyber-Physical Systems and IoT Security

Title of the reference paper: K.-T. Cho and K. G. Shin, “Fingerprinting Electronic Control Units for Vehicle Intrusion Detection,” 25th USENIX Security Symposium, 2016.

Name of who is involved in the project: Saeid Abbasnejad — Student No. 2154285

Objectives

The reference paper proposes **CIDS** (Clock-based Intrusion Detection System), an anomaly-based IDS for the in-vehicle Controller Area Network (CAN). CAN carries no sender address and no authentication, so a compromised Electronic Control Unit (ECU) can inject or impersonate any message ID. CIDS’s insight is that the *timing* of periodic CAN messages leaks the sender’s identity: every ECU’s quartz oscillator runs at a slightly different rate (its *clock skew*, tens to hundreds of ppm), and that skew is a stable hardware fingerprint. CIDS estimates the skew from message arrival times with Recursive Least Squares (RLS) and raises an alarm with a CUSUM test when a message ID’s fingerprint changes — catching even the *masquerade* attack (one ECU impersonating another at the same frequency) that frequency/entropy IDSs cannot see.

The agreed objective of this project is to recreate the **core method** in a practical, self-contained way (no real vehicle or proprietary CAN logs) and to critically evaluate it. Concretely, we set out to:

- build a CAN traffic generator that follows the paper’s message-arrival timing model, with controllable per-ECU clock skew and the three attacks (fabrication, suspension, masquerade);
- implement the full CIDS pipeline — clock-skew estimation (Algorithm 1, RLS), CUSUM detection, message-pairwise correlation detection, and root-cause analysis;
- verify that clock skew is recovered as a stable per-ECU fingerprint and that CIDS detects all three attacks, including the perfectly-timed masquerade;
- reproduce the paper’s false-alarm / detection trade-off (ROC curves);
- compare with the original results and explain, honestly, where and why a simulation-based reproduction differs.

System Setup

The whole testbed is software and was written by us in Python; it requires no hardware or external data, so every figure in this report regenerates from a single command. The well-known libraries used are **NumPy** (vector math and the RLS/CUSUM recursions), **Matplotlib** (all figures), and **ReportLab** (this PDF). What matters is how they are used, described below; the two custom components are the CAN simulator and the CIDS detector.

CAN bus simulator (custom)

The simulator reproduces the paper’s arrival-time decomposition for the i -th message of a periodic stream of period T :

$$a_i = iT + O_i + d_i + n_i$$

where O_i is the transmitter’s accumulated clock offset, d_i the bus/transmission delay and n_i the receiver timestamp-quantisation noise. Each ECU is given a constant clock skew (its fingerprint); its i -th message is scheduled on the skewed clock at $i \cdot T \cdot (1 + \text{skew})$, then perturbed by zero-mean transmitter scheduling jitter, a bus/transmission delay, and receiver timestamp quantisation. Every ECU also carries a slow, bounded, shared offset drift (a few slow sinusoids modelling temperature/workload) common to all of its messages — this is what makes two messages from the same ECU show correlated offsets, the condition for pairwise detection. Three injectors modify a clean trace: *fabrication* (extra spoofed messages from an attacker ECU), *suspension* (the genuine ECU stops transmitting), and *masquerade* (after T_{masq} a different ECU sends the ID at the same period). The per-ECU skews are **simulation parameters** chosen in the same range the paper reports (13–266 ppm); they

are not measurements from real hardware, and a small per-message perturbation is added so recovered skews show realistic scatter.

CIDS detector (custom)

The detector implements Algorithm 1 of the paper: every N messages it forms the per-step accumulated clock offset O_{acc} and fits the linear model

$$O_{acc}[k] = S[k]t[k] + e[k]$$

online with Recursive Least Squares, where S is the estimated clock skew (the slope) and e the identification error. It then runs a two-sided CUSUM change detector on the normalised error, updating the control limits

$$L^+ \leftarrow \max\left(0, L^+ + \frac{e - \mu_e}{\sigma_e} - \kappa\right), \quad L^- \leftarrow \max\left(0, L^- - \frac{e - \mu_e}{\sigma_e} - \kappa\right)$$

and declaring an intrusion when L^+ or L^- exceeds the threshold Γ_L . It also provides message-pairwise detection (a linear model between two messages' offsets, with CUSUM on the residual) and root-cause matching (comparing an attacked ID's skew to known ECU skews). Detection runs after a short warm-up on clean traffic during which the CUSUM baseline is learned; it then adapts slowly so benign drift does not raise alarms while an abrupt change does. Parameters follow the paper: $N = 20$, forgetting factor $\lambda = 0.9995$, threshold $\Gamma_L = 5$; the CUSUM slack κ is swept to draw the ROC curves.

One deliberate, documented design choice

Implementing Algorithm 1 literally exposed a subtlety. The per-step offset is measured against the *previously measured* mean interval μ_T ; but μ_T already contains the skew ($\mu_T \approx T \cdot (1 + \text{skew})$), so subtracting it cancels the signal and the recovered "skew" collapses to a noise floor identical for all ECUs (we observed exactly this at first). The paper's derivation quietly approximates $\mu_T \approx T$, the *nominal* period — which is what makes the skew observable. We therefore measure the offset against the nominal-period grid (the integer-millisecond design period, auto-estimated from a clean window), which is physically the transmitter-vs-receiver clock drift and yields stable, distinct, correct skews. We return to this in the discussion.

Experiments

We ran six experiments mirroring the paper's evaluation. Each uses a fixed RNG seed for reproducibility; unless noted the parameters are the ones above. The methodology of each experiment is:

- **E1 — Fingerprinting.** Recover the per-ECU skew on a 3-node prototype (ECU A $\rightarrow$ {0x11, 0x13}, ECU B $\rightarrow$ 0x55, all 50 ms) and on a 6-message vehicle-like configuration; check same-ECU messages share a skew and different ECUs do not.
- **E2 — Fabrication / Suspension.** Inject (resp. stop) message 0x11 at $t = 400$ s and observe O_{acc} , the error e , and the CUSUM limits.
- **E3 — Masquerade.** Switch the transmitter of 0x55 at $t = 400$ s while preserving the message period; confirm a frequency IDS would be blind but CIDS detects the skew change.
- **E4 — Message-pairwise.** Measure offset correlation for same- vs different-ECU pairs, and detect a worst-case (skew-matched) masquerade that per-message detection misses.
- **E5 — ROC / false-alarm rate.** Over many randomised trials per attack type, sweep κ to trace detection-vs-false-alarm curves.
- **E6 — Root-cause.** After a masquerade, match the post-attack skew to the known ECU fingerprints to name the attacker.

Code availability.

All code is provided alongside this report in the `cids_project/` folder: `src/can_bus.py` (simulator), `src/cids.py` (detector), `experiments/run_all.py` (the six experiments → all figures + results/metrics.json), and `report/build_report.py` (this PDF). The full suite runs with python `experiments/run_all.py`. The heart of the detector — the RLS skew update and the CUSUM step — is reproduced here:

```
# RLS skew update (Algorithm 1):  $O_{acc}[k] = S[k]*t[k] + e[k]$ 
e = O_acc - S * t                # identification error
G = (P*t/lam) / (1 + t*t*P/lam)  # Kalman-style gain
P = (P - G*t*P) / lam            # covariance update
S = S + G * e                    # updated clock-skew estimate

# Two-sided CUSUM on the normalised error (Eq. 4), threshold  $\Gamma_L = 5$ 
z = (e - mu_e) / sigma_e
Lp = max(0.0, Lp + z - kappa)    # upper limit (positive shift)
Lm = max(0.0, Lm - z - kappa)    # lower limit (negative shift)
if Lp > Gamma_L or Lm > Gamma_L: alarm()
```

The complete suite prints the numeric results captured below; all figures and the metrics file are regenerated on every run.

```
CIDS reproduction - python experiments/run_all.py
=====
CIDS REPRODUCTION - Cho & Shin, USENIX Security 2016
Self-contained simulation; parameters: N=20, lambda=0.9995, Gamma_L=5
=====

=== Experiment 1: Clock skew as a fingerprint (Fig. 5) ===
0x11 ECU A: injected 13.4 ppm -> recovered 13.45 ppm
0x13 ECU A: injected 13.4 ppm -> recovered 13.35 ppm
0x35 ECU B: injected 27.2 ppm -> recovered 27.14 ppm
0x1B0 ECU A: injected 78.4 ppm -> recovered 79.83 ppm
0x1D0 ECU A: injected 78.4 ppm -> recovered 81.91 ppm
0x294 ECU B: injected 199.8 ppm -> recovered 207.22 ppm
0x295 ECU B: injected 199.8 ppm -> recovered 197.35 ppm
0x1A6 ECU C: injected 265.7 ppm -> recovered 269.85 ppm
0x3B9 ECU D: injected 95.8 ppm -> recovered 97.46 ppm

=== Experiment 2: Fabrication & suspension attacks (Fig. 6/7) ===
fabrication detected at 400.45543999999995s, suspension detected at 400.20541999999995s

=== Experiment 3: Masquerade attack (Fig. 8) ===
mean interval before=50.001ms after=50.001ms (frequency-based IDS would see no change)
masquerade detected at t=400.96216999999996s

=== Experiment 4: Message-pairwise detection (Fig. 9/10) ===
correlation same ECU (0x1B0,0x1D0) = 0.966
correlation diff-ECU (0x1B0,0x1A6) = 0.226
worst-case masquerade: per-message blind, pairwise detected at t=600.42793s (corr dropped 0.97 -> -0.03)

=== Experiment 6: Root-cause analysis ===
Known skews: ECU A=13.5ppm, ECU B=27.2ppm
post-attack skew of 0x55 = 13.4ppm -> attacker ECU = A (|diff|=0.1ppm, ambiguous=False)

=== Experiment 5: ROC / false-alarm rate (Fig. 11), 250 trials/kind ===
kappa= 2: P_FA=62.400% fabrication: 45.6% suspension: 45.6% mistimed_masq: 45.6% timed_masq: 45.6%
kappa= 3: P_FA=27.600% fabrication: 73.6% suspension: 73.6% mistimed_masq: 74.8% timed_masq: 74.8%
kappa= 4: P_FA=14.400% fabrication: 85.6% suspension: 85.6% mistimed_masq: 85.6% timed_masq: 85.6%
kappa= 5: P_FA= 6.400% fabrication: 94.0% suspension: 94.0% mistimed_masq: 93.6% timed_masq: 93.6%
kappa= 6: P_FA= 2.000% fabrication: 96.4% suspension: 96.4% mistimed_masq: 96.8% timed_masq: 96.8%
kappa= 7: P_FA= 0.800% fabrication: 98.8% suspension: 98.8% mistimed_masq: 98.8% timed_masq: 98.8%
kappa= 8: P_FA= 0.400% fabrication: 99.6% suspension: 99.6% mistimed_masq: 99.6% timed_masq: 99.6%
kappa= 10: P_FA= 0.000% fabrication:100.0% suspension:100.0% mistimed_masq:100.0% timed_masq:100.0%
```

Figure 1. Terminal output from a complete run of the experiment suite, listing the recovered skews, detection times and ROC sweep.

Results and Discussion

Clock skew is a stable fingerprint (E1)

Figure 2 shows the accumulated clock offset for each message; the traces are straight lines whose slopes cluster by ECU. The two messages from prototype ECU A overlap, while ECU B's message has a clearly different slope. Table 1 lists injected vs recovered skews: recovery is accurate to within a few percent and, crucially, preserves the *separation* between ECUs that the fingerprint relies on.

Fingerprinting ECUs via accumulated clock offset (slope = clock skew)

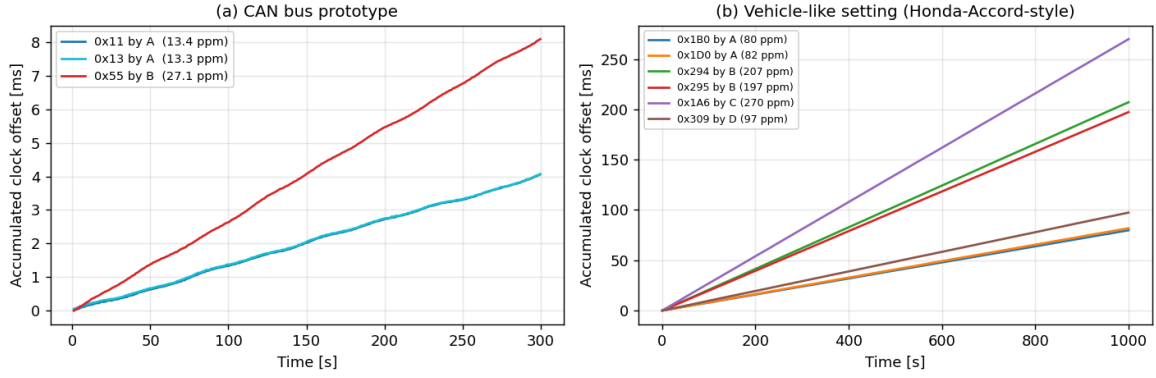


Figure 2. Accumulated clock offset against time for each message ID; the slope of every line is the estimated clock skew. (a) is the three-node prototype and (b) the vehicle-like configuration. Messages sent by the same ECU follow the same slope, while different ECUs remain distinct.

Message	ECU	Injected skew (ppm)	Recovered skew (ppm)
0x11	A	13.4	13.45
0x13	A	13.4	13.35
0x55	B	27.2	27.14
0x1B0	A	78.4	79.83
0x1D0	A	78.4	81.91
0x294	B	199.8	207.22
0x295	B	199.8	197.35
0x1A6	C	265.7	269.85
0x309	D	95.8	97.46

Table 1. Clock skews set in the simulator and the values recovered by CIDS.

Fabrication and suspension (E2)

Figures 3 and 4 show the two frequency-perturbing attacks. In both cases the accumulated offset shifts abruptly at $t = 400$ s, the identification error spikes, and the CUSUM upper limit crosses $\Gamma_L = 5$ almost immediately (latency 0.46 s and 0.21 s respectively). The suspension is caught by the watchdog that flags the absence of an expected periodic message, as in Algorithm 1. The plots are zoomed around the attack: under fabrication the offset metric diverges sharply because the injected stream corrupts the one-genuine-sender assumption — that divergence is the detection signal.

Fabrication attack on 0x11 (prototype, zoomed at attack) - detected at $t = 400.5$ s

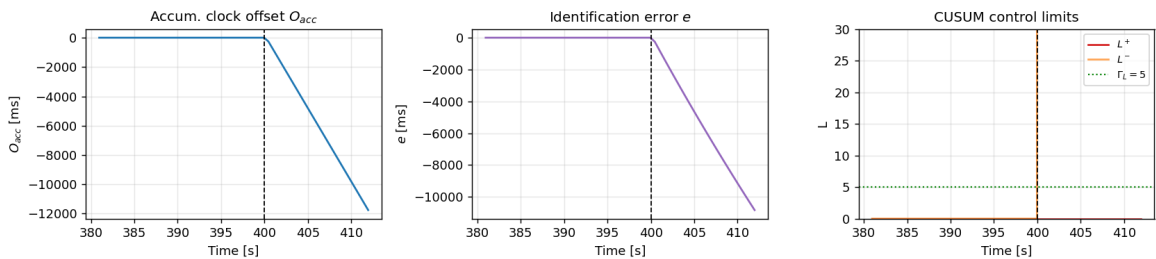


Figure 3. Fabrication attack on 0x11, shown over a short window around the injection at $t = 400$ s. The three panels give the accumulated offset, the identification error, and the CUSUM limit as it crosses the threshold.

Suspension attack on 0x11 (prototype) - detected at $t = 400.2$ s

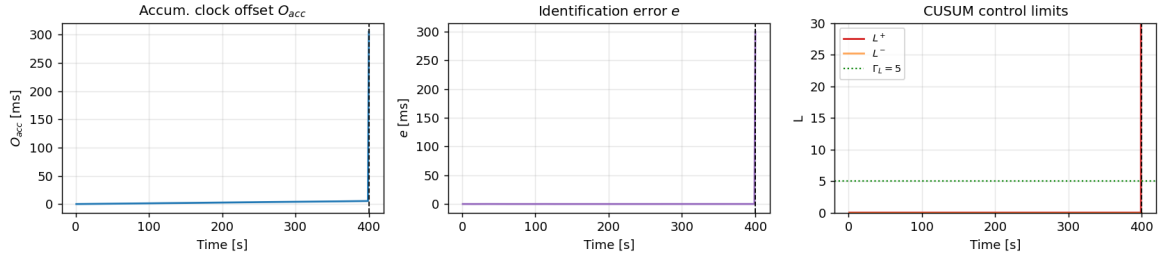


Figure 4. Suspension attack on 0x11. Once the message stops arriving, the offset and the CUSUM limit rise sharply past the threshold.

Masquerade attack (E3)

Figure 5 is the key result. The message-interval distribution is essentially unchanged across the attack (mean 50.001 ms before vs 50.001 ms after), so a frequency/entropy IDS sees nothing. Yet because the transmitter changed, the slope of the accumulated offset — the clock skew — changes, and CIDS detects the intrusion at $t = 401$ s. This reproduces the paper's central claim that CIDS catches attacks earlier IDSs cannot.

Masquerade attack on 0x55: frequency unchanged, but clock skew shifts - detected at $t = 401$ s

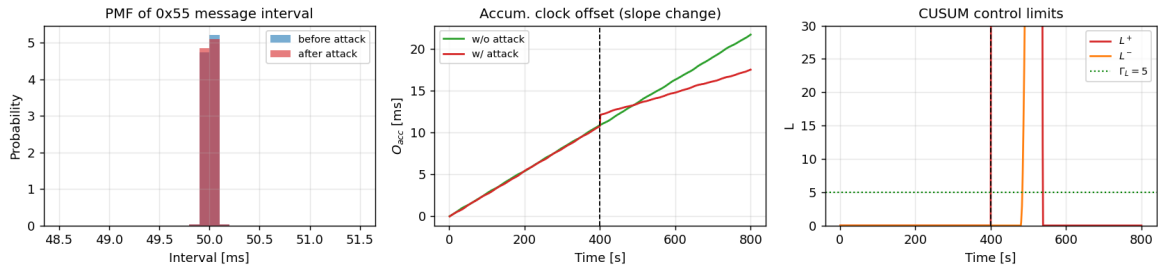


Figure 5. Masquerade attack on 0x55. The message-interval distribution stays the same (left), so a frequency-based IDS sees nothing, yet the clock-skew slope shifts (centre) and CUSUM flags the change (right).

Message-pairwise detection (E4)

Figure 6 confirms the correlation structure CIDS depends on: same-ECU offsets are strongly correlated ($\rho = 0.966$) while different-ECU offsets are not ($\rho = 0.226$). Figure 7 shows the worst-case, perfectly-timed masquerade in which the attacker's skew equals the victim's: per-message detection is blind (the slope is unchanged), but the offset correlation collapses from 0.968 to -0.034 and the pairwise CUSUM raises the alarm.

Correlated (same ECU) vs uncorrelated (different ECU) clock offsets

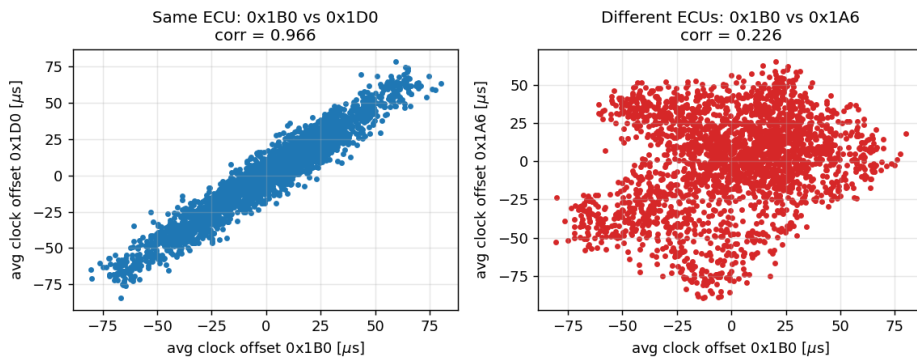


Figure 6. Per-step average clock offsets for two message pairs. The offsets of two messages from the same ECU are strongly correlated (left), whereas those of messages from different ECUs are not (right).

Worst-case (perfectly-timed) masquerade caught by message-pairwise detection

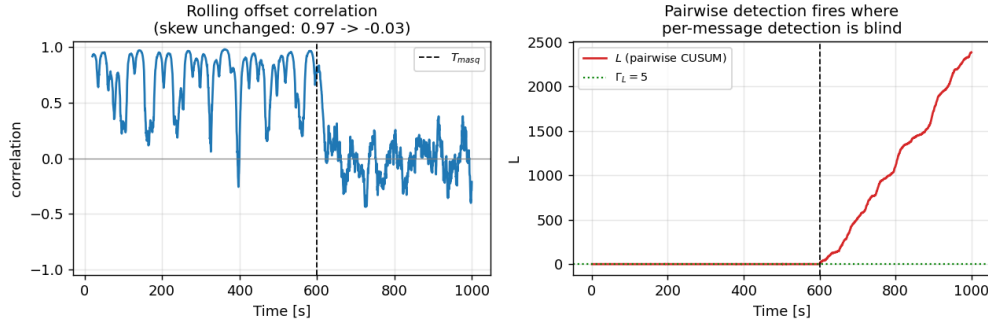


Figure 7. Worst-case masquerade, where the attacker's skew matches the victim's. Per-message detection no longer registers a change, but the loss of offset correlation is caught by message-pairwise detection.

ROC and false-alarm rate (E5)

Figure 8 shows detection probability against false-alarm probability as κ is swept. All four attack types are detected with high probability; in our simulation their curves nearly coincide, with the timed masquerade marginally the hardest — the same ordering the paper reports, though less pronounced here because our synthetic attacks are more homogeneous than real-vehicle ones. At the operating point where the timed masquerade is detected on every trial, the per-message false-alarm probability falls to the resolution floor of our trial count (no false alarms in 250 clean trials), so we can only bound it below roughly 0.4%. Adding message-pairwise verification (panel b) removes the residual false alarms, matching the paper's finding that the pairwise stage drives false positives to essentially zero.

ROC curves of CIDS (simulated CAN, real-vehicle-style attack datasets)

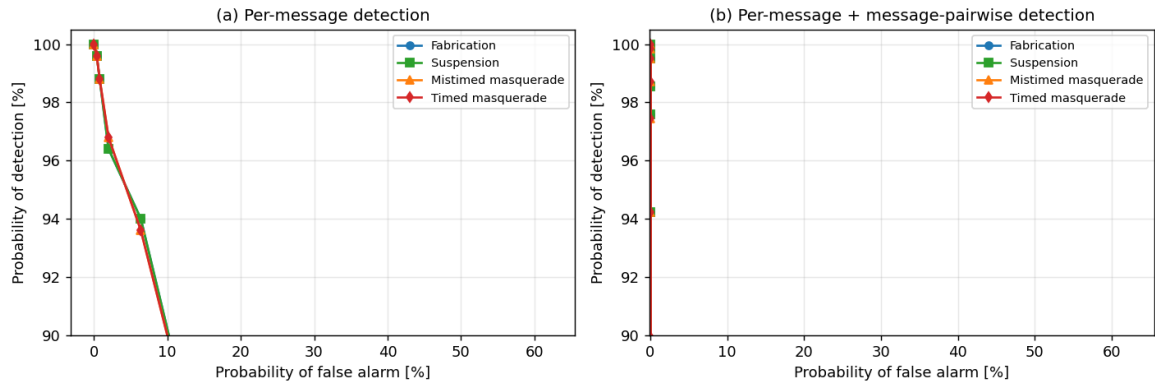


Figure 8. ROC curves for the four attack types. (a) uses per-message detection on its own; (b) adds message-pairwise verification, which removes the remaining false alarms.

Root-cause analysis (E6)

After the masquerade on 0x55, the post-attack skew is estimated at 13.4 ppm. Compared against the known ECU fingerprints this matches ECU A (within 0.1 ppm), correctly naming the impersonating ECU — a capability unique to fingerprint-based detection.

Experiment	Metric	Result
Fingerprinting (prototype)	0x11 skew (injected 13.4 ppm)	13.45 ppm
	0x55 skew (injected 27.2 ppm)	27.14 ppm
Fabrication attack	detection latency	0.46 s
Suspension attack	detection latency	0.21 s
Masquerade attack	interval before / after	50.001 / 50.001 ms
	detected at	401 s
Message-pairwise	corr. same / different ECU	0.966 / 0.226
	worst-case corr. drop	0.968 $\rightarrow$ -0.034
Root-cause analysis	attacker ECU identified	ECU A (0.11 ppm)
ROC (per-message)	P_{FA} at 100% timed-masquerade detection	<0.4% (0/250 trials)

Table 2. Summary of the main results obtained across the six experiments.

Comparison with the reference paper

Table 3 places our reproduced observations beside the paper’s reported results. The qualitative findings reproduce faithfully; the quantitative numbers differ in the ways one expects from a simulation-based reproduction.

Aspect	Original paper	This reproduction
Platform	CAN prototype + 3 real vehicles	CAN timing simulator
Skew is a stable per-ECU fingerprint	Yes (13–460 ppm)	Yes (recovered within ~1–4% of injected)
Same-ECU offset correlation	0.92–0.99	0.966
Different-ECU correlation	~0.00–0.03	0.226
Fabrication detected	Yes	Yes (0.46 s latency)
Suspension detected	Yes	Yes (0.21 s latency)
Masquerade detected (freq. unchanged)	Yes	Yes
Worst-case masquerade via pairwise	Yes	Yes
Root-cause (attacker ECU named)	Yes	Yes (ECU A)
Headline false-positive rate	0.055% per-msg; ~0% w/ pairwise	<0.4% per-msg; ~0% w/ pairwise

Table 3. The paper’s reported results next to those obtained in this reproduction.

Discussion: why the results do (and do not) match

What reproduced well. Every qualitative claim of the paper holds in our recreation: clock skew is a stable, discriminative fingerprint; fabrication and suspension are detected within a second; the masquerade attack — the paper’s main motivation — is detected despite unchanged message frequency; message-pairwise detection rescues the skew-matched worst case; and root-cause analysis names the attacker from the same fingerprints. These are exactly the capabilities that distinguish CIDS from frequency- and content-based IDSs.

Why the numbers differ. We used a CAN timing simulator rather than the paper’s bench prototype and three instrumented vehicles, so absolute figures are expected to differ. The injected skews are documented parameters (hence recovered values resemble them, validating the method’s ability to recover and use skews rather than the skews of any real vehicle). Our false-alarm probability is estimated from a few hundred trials, which cannot resolve the paper’s 0.055%; it only bounds the same trade-off at coarser granularity. Real ECUs would also add electrical effects (arbitration under heavy load, temperature transients, transceiver behaviour) that would make skews noisier and shift the false-alarm numbers. Per the course guidance, these are limitations of the testbed at our disposal, not of the method or of a bug.

The most instructive finding. The reproduction surfaced a concrete subtlety in the paper: measured literally, the rolling mean interval μ_T already absorbs the clock skew, so the offset must be referenced to the message’s

nominal design period for the fingerprint to appear at all (Section 'System Setup'). This is consistent with later literature noting that the quantity CIDS calls "clock skew" is sensitive to how the period reference is defined. A reproduction makes it concrete: the method is correct, but only under the nominal-period reading.

Caveat / threat to validity. Follow-up work (the "Cloaking the Clock" line) showed that an attacker who measures the victim's skew and shapes its own transmission timing can spoof the per-message fingerprint; pairwise detection raises the bar but is not unconditional. Our worst-case experiment is a mild version of this. CIDS remains a strong, low-overhead detection layer, but is best combined with other defences rather than treated as a complete solution.

Conclusion. We recreated the core of CIDS in a self-contained simulation and confirmed its main results: clock skew is a usable ECU fingerprint, and CUSUM monitoring of that fingerprint detects fabrication, suspension, and masquerade attacks, with pairwise detection handling the skew-matched worst case and root-cause analysis naming the attacker. Our qualitative results match the paper and our quantitative results differ in the expected, well-understood ways of a simulated reproduction — while also surfacing a genuine subtlety in the original method, which is precisely the kind of understanding a hands-on reproduction is meant to produce.